BoxOffice Brain

A grounded cinema-analytics agent on ClickHouse

Dipankar Sarkar
github.com/doom2quake

Abstract

An analytics team asks a question in plain English and wants a number it can act on, not a plausible guess. BoxOffice Brain answers with a Gemini/ADK agent on Vertex AI that writes read-only ClickHouse SQL and runs it through ClickHouse's own Model Context Protocol server at request time. Every result carries a grounding trace (the exact SQL, rows and bytes scanned, latency), every quantitative claim traces to a retrieved row, and the agent abstains with a stated data gap when the rows cannot support the question. A failed query is reported as a failure rather than as an empty result. Two properties are measured rather than asserted: the same statement executed on three transports (the official MCP server, ClickHouse HTTP, and an embedded engine over a committed extract of the same tables) returns identical values to within 10^{-9}, and a data-scope guardrail refuses ClickHouse table functions that a read-only screen alone would let through.

1 Problem

Text-to-analytics agents are easy to make impressive and hard to make trustworthy. The failure mode that matters is not a wrong join, it is a confident answer that no row supports. A reader cannot tell a grounded "War fell 0.78 rating points" from a hallucinated one, and a summary over a stale extract is already wrong the moment the warehouse moves.

A second failure mode is subtler and appears once an agent writes its own SQL. A read-only screen accepts SELECT * FROM url('http://…'), which is a single SELECT statement that reads the open internet, and SELECT * FROM file('/etc/passwd'), which reads the local disk. A prompt-injected question is exactly how such a statement gets written. Read-only is a necessary check and not a sufficient one.

We therefore want three properties: every quoted number traces to a row that came back at request time; the agent refuses, and says what it would need, when the rows do not support an answer; and the set of data the generated SQL can reach is bounded by an explicit allowlist rather than by the service account's incidental privileges.

2 Design

BoxOffice Brain is a Gemini/ADK supervisor over three named skills assembled on agent-core, a reusable ADK core. At request time the supervisor runs Gemini (gemini-2.5-flash) on Vertex AI and delegates in order:

- Planner calls describe_tables to read the schema it is permitted to see, then states the metric, the comparison, the dimensions, and the second signal to check. No SQL yet.
- Analyst writes read-only ClickHouse SQL and runs it via run_clickhouse_sql, returning rows plus a grounding trace per query.
- Answer Writer synthesises a decision-ready answer from the retrieved rows only, or abstains.

With GOOGLE_GENAI_USE_VERTEXAI=True and a Google Cloud project, Gemini itself writes the SQL and issues the MCP run_query call; a captured end-to-end Vertex run is in docs/gemini-run.txt, showing the planner output, the SQL Gemini wrote, the rows the MCP server returned, and the agent-core action limiter capping further queries. The keyless --no-llm router below is the reproducible, no-credential path used by the tests and figures; both paths execute the same read-only SQL against the same ClickHouse, and neither quotes a number that did not come back in a row.

2.1 The MCP hop is load-bearing

With BOX_TRANSPORT=mcp, run_clickhouse_sql executes by launching mcp-clickhouse, the MCP server ClickHouse publishes, over stdio and calling its run_query tool. The guardrails sit in front of that hop and the official server does the work behind it. If the server cannot start, the transport returns status: error; it does not fall back to in-process tools while still reporting success. One session is held open on a background event loop, because launching the server per query costs seconds and would misrepresent ClickHouse's latency.

The endpoint defaults to the public ClickHouse SQL playground with the read-only demo user, the same endpoint the official server defaults to, so the integration is

reproducible without a funded key. The corpus is IMDb catalogue data: 388,269 films and 3.4M roles.

3 Grounding, abstain, and failure

The hero mechanism is the contract on the query tool. Every call returns a status and a structured grounding block:

```
{"status": "ok", "rows": [...],
 "grounding": {"sql": "...", "note": "...",
   "rows_read": N, "bytes_read": B,
   "elapsed_ms": T, "source": "clickhouse_mcp",
   "transport": "mcp"}}
```

status == "ok" is the only value from which rows may be read. This distinction is load-bearing: an earlier build treated any result as a row list, so a simulated ClickHouse timeout was reported as the finding "no titles matched". An operational failure now surfaces as an execution error, and the answer says so.

Three outcomes are therefore distinguishable: a grounded answer, an abstention because the query ran and returned nothing that answers the question, and an execution error. A fourth case is refused before any query runs: a question that maps to no named analytical template, which the agent answers by listing what it can ground.

4 Guardrails

Four gates run before a byte leaves the process, and each decision is appended to the run's audit trail in the state store, so the interface displays the recorded outcome rather than a decorative tick.

1. READ_ONLY_SQL: a single SELECT/WITH, no writes or DDL.
2. QUERY_SCOPE: comments are stripped, string literals masked, and the statement refused if it calls a ClickHouse table function (url, file, s3, remote, mysql, and 25 others), carries a SETTINGS clause or INTO OUTFILE, or reads any table outside an explicit allowlist. Common table expressions are resolved rather than allowlisted, and an identifier the scanner cannot resolve is refused.
3. ACTION_LIMITER: per-run and per-hour caps, keyed off a ContextVar so concurrent runs cannot spend each other's budget.
4. Server-side bounds: readonly=1, max_result_rows, max_result_bytes and max_execution_time, plus a bounded client read, so an unbounded generated query is stopped by ClickHouse rather than by the client exhausting memory after downloading the whole result.

5 Results

5.1 Cross-engine parity

The offline transport is an embedded SQLite engine over a 11,239-row extract of the same imdb.movies and imdb.genres tables, attached under the schema name imdb so the identical statement text runs on both. The analytical templates are written in the intersection of ClickHouse SQL and SQLite SQL (CASE WHEN rather than countIf), which makes a real comparison possible.

Running the hero query (mean rating per genre, 1999 against 1998, genres with at least 20 rated titles in each year) on all three transports and comparing every returned cell:

transport	rows	engine ms	rows read
offline	18	17.7	11,239
http	18	387.1	783,388
mcp	18	1,032.3	n/a

All three agreed on all 18 rows and every numeric cell to within 10^{-9}. Reaching that tolerance required one deliberate fix: ClickHouse stores rank as Float32 and promotes it to Float64 when averaging, so the loader round-trips each value through a 32-bit float. Without it the engines diverge in the seventh significant digit.

The latencies are not a benchmark and should not be read as one. The offline engine reads 11,239 rows because that is the whole extract; the warehouse reads 783,388 for the same answer. The MCP figure is the tool call on a warm session, and the official server's run_query returns columns and rows only, so scan statistics are unavailable on that path and are displayed as n/a rather than estimated.

5.2 Tests

35 keyless tests and 4 opt-in live tests. The keyless set pins the read-only screen, the data-scope allowlist against five table-function payloads and a cross-database read, a comment-hidden payload, per-context run isolation, the offline engine evaluating predicates (WHERE 1=0 returns zero rows) and reporting invalid SQL as an error, question routing to distinct templates, abstention on both empty rows and unsupported questions, a failed query not being read as absent data, the correlated signal being scoped to the cohort the headline named, run finalisation with a persisted guardrail

trail, and fail-closed MCP behaviour. The live set starts ClickHouse's own MCP server, checks its advertised tool names, and compares live rows against the offline snapshot.

6 Limitations

Gemini and ADK run at request time on Vertex AI: with a Google Cloud project the supervisor drives Gemini, which writes the SQL and calls the MCP run_query tool (docs/gemini-run.txt). The parity and latency figures in this paper, and the tests, come instead from the keyless grounded router, which maps a question to one of four named templates and abstains otherwise. That router is a deliberate downgrade for reproducibility, not natural language understanding, so anyone can reproduce the numbers without a key; it is the source of the reported cells, and we make no quantitative claim about model behaviour over a prompt set.

The live endpoint is the public ClickHouse SQL playground with the read-only demo user, not a private ClickHouse Cloud service, and the corpus is IMDb catalogue data; the credentialled path to a paid cluster is therefore untested. The guardrails are regex-level checks over comment-stripped SQL, not a parser; an exotic ClickHouse construct could in principle be shaped past them, which is why server-side bounds and a read-only account also apply. The abstain rule bounds hallucination at the retrieved-rows boundary but does not verify that the SQL answers the question asked. The offline extract covers 1998 and 1999 only. There is no holdout evaluation over a question distribution; cross-engine parity is a correctness check on one query, not a measure of answer quality.

7 Conclusion

An analytics agent can put its evidence on the table, refuse when the table is empty, say so when the query failed, and be prevented from reading data it was never meant to see. BoxOffice Brain quotes a number only when a ClickHouse row supports it, shows the SQL and the scan behind every answer, and can be checked against the live warehouse by anyone, without a key.